UAB
Universitat Autònoma de Barcelona

Internship Report

Master's Degree in Modelling for Science and Engineering

Universitat Autònoma de Barcelona

Author: Ferran Mirabent Rubinat (1528268)
Host institution: Barcelona Supercomputing Center (BSC), Computational Earth Sciences (CES) group, Artificial Intelligence team
Company tutor: Oriol Tintó Prims (BSC)
Academic tutor: Silvia Cuadrado Gavilan (UAB)
Period: 16 February 2026 to 30 June 2026

---

1 Introduction

This report documents the internship done as part of the Master's Degree in Modelling for Science and Engineering at the Universitat Autònoma de Barcelona (UAB). The internship took place at the Barcelona Supercomputing Center (BSC), within the Artificial Intelligence team of the Computational Earth Sciences group. It was supervised by Oriol Tintó Prims as the company tutor at BSC and by Silvia Cuadrado Gavilan as the academic tutor at UAB. The work done during the stay is parallel to the work for my Final Master's Thesis, *Performance Optimization of the AI4Land Training Pipeline on MareNostrum 5*, and this report describes the stay itself: the host, the team, the tasks, and what I took from the experience.

2 Company Details and Assigned Tutor

The host institution is the Barcelona Supercomputing Center located in Barcelona. Within BSC, I was hosted by the Artificial Intelligence team of the Computational Earth Sciences (CES) group, part of the Earth Sciences Department. The group develops climate and Earth-system models and machine-learning methods applied to Earth-science data. My day-to-day work was on the AI4Land project, which the group runs inside 2 European Horizon Europe programmes, CONCERTO and TerraDT.

The tutor assigned by the company was Oriol Tintó Prims, a researcher at BSC who directed the internship, set its objectives, and reviewed its progress. The academic side was tutored by Silvia Cuadrado Gavilan at UAB.

3 Nature of the Company

BSC is the national supercomputing centre of Spain. It was founded in 2005 and is run as a public consortium of the Spanish government, the Generalitat de Catalunya, and the

Universitat Politècnica de Catalunya (UPC). Its purpose is twofold: to operate large-scale computing machines for the Catalan and European research communities, and to carry out its own research in computer sciences, life sciences, Earth sciences, and engineering applications.

The centre operates MareNostrum 5, one of the most powerful supercomputers in Europe and a pre-exascale system of the European High-Performance Computing Joint Undertaking (EuroHPC JU). A supercomputer is a large cluster of compute nodes joined by a fast network, built to run scientific calculations that do not fit on a single workstation; high-performance computing (HPC) is the umbrella term for this class of machine and the work run on it. The part of MareNostrum 5 that this internship used is its accelerated partition, built around NVIDIA H100 graphics processing units (GPUs), the processors that modern deep-learning training relies on. BSC allocates time on these machines to research groups across Europe, so the efficiency of any pipeline that runs on them has a direct cost in machine time, in queue waiting, and in the pace of the science it supports.

4 Teamwork

I joined an existing team and an existing codebase, so the work was collaborative from the start. I worked most closely with Amirpasha Mozaffari and Marina Castaño, the main developers of AI4Land, who shared the running pipeline with me and answered my questions about the model, the data, and the cluster throughout the stay. The team is led by Amanda Duarte. Next, I worked closely with Joan Vedri, who builds the project's datasets: he created the various data stores I tested and reviewed the code that merges them into the final one. The single most effective change of the internship, a new storage layout for the training data, was designed together with him, with me measuring where the cost was and which layout would remove it, and him building the store that put it into practice. Jordi Varela helped me get the profiling started, making the first attempts at the Nsight Systems traces, but I was the one who got them to actually reveal the problem and learned to drive the profiling tools properly.

Working inside a shared HPC system also forced another kind of teamwork. MareNostrum 5 is used by hundreds of jobs at once, and 2 measurement jobs reading the same data at the same time distort each other's timings. Early in the internship I ran 2 profiling jobs concurrently and got misleading numbers from it. After that I coordinated runs so that no 2 profiling jobs competed for the same storage, and the rule was written into the team's working notes so others would not repeat the mistake. Beyond the measurements, I shared profiling findings with the team as the work progressed, so that the conclusions and the new pipeline were understood and adopted rather than handed over at the end.

5 Tasks Carried Out

The internship and the thesis are 2 views of the same project. The thesis presents the in-depth performance analysis that locates the training bottleneck and explains it. This section describes the engineering work of the stay, which the thesis only summarizes: making the project reproducible and quick to iterate on, writing the code that profiles the training, and carrying the resulting fix through to the full production dataset. That last part is not covered by the thesis.

Migrating the environment to uv. The project's environment was managed with Conda, a tool that builds an isolated set of packages and resolves their versions with a solver. It had 3 properties that got in the way. Conda stores a full, separate copy of every package for each environment, so on a shared filesystem with per-user storage quotas the team could not afford 1 environment per branch and shared a single one across all of them. Because that environment was shared, a change to a dependency on 1 branch became active for everybody, and undoing it to reproduce an older result was not a self-contained operation. And the solver is slow: a clean build of the environment took several minutes. I migrated the project to uv, a newer manager built for speed. uv keeps 1 copy of each downloaded package in a shared cache and links it into every environment that needs it, so environments that share a package share its files on disk instead of duplicating them, and a per-branch environment costs only its differences from the cache. uv also writes a lockfile that pins every direct and indirect dependency to an exact version with a checksum, so the same environment can be rebuilt identically on another machine or at a later date. And its solver runs in seconds. The migration was more than a swap of tools. I wrote the standard project-description file the wider Python community uses, reorganized the source into an installable package so the code can be run from anywhere on the cluster instead of by relative path from 1 copy of it, and rewrote the launch scripts to import from that package. In the same pass I pulled many file paths and settings out of the scripts, where they had been written in by hand, and moved them into configuration, so that a run is now described by its configuration rather than by edited code.

Choosing how to provide CUDA. Training on the GPUs relies on CUDA (Compute Unified Device Architecture), NVIDIA's platform for general-purpose computation on its graphics hardware. CUDA can be provided in 2 ways: the cluster offers a system-wide CUDA toolkit that you load as an environment module, or the PyTorch package can carry its own matching CUDA runtime bundled inside it. I tested both on the H100 partition for correctness and for speed. They ran equally fast, with the PyTorch-bundled runtime as quick as the system module or slightly quicker. I chose the bundled PyTorch version, because it is simpler in every respect: the environment is self-contained, it does not depend on which system module happens to be loaded, and upgrading it is a single change to the project's dependencies.

Code style with ruff. I added ruff, a combined linter and formatter, to the project. A linter flags likely mistakes and stylistic problems; a formatter rewrites the code into one canonical style. ruff does both from a single, fast tool, and runs over the whole project in well under 1 second. It is enforced by a pre-commit hook, a small check that runs automatically before each recorded change and refuses the change if the style is not clean, so the only style ever committed is the project's. The point is to keep each change easy to review. A change that mixes a real edit with incidental reformatting is hard to read; one that contains only the real edit can be read at a glance. A uniform style across the codebase also makes outliers, which are often where bugs hide, easier to spot.

Test gates: smoke and parity. I wrote 2 small test scripts that run on the cluster and act as gates before any change is merged. The smoke test runs a short training epoch on 1 GPU and checks that the loss goes down and that nothing raises an error. It catches gross breakage: a refactor that breaks the data loader, a configuration change that silently switches off an input, a tensor-shape mismatch from a layout change. It

finishes in a few minutes. The parity test draws a fixed set of samples from 2 versions of the pipeline and checks that the input tensors they produce are bit-identical, or, when floating-point summation order makes exact equality impossible, that every per-sample statistic agrees to within a strict tolerance. It runs before any speed comparison between 2 data layouts, because a layout that runs faster but feeds the model different numbers is not a fair comparison. Every change to the data path in this work was accepted only after the parity test passed.

Profiling the training. To see where the training time went, I instrumented the training loop for NVIDIA Nsight Systems, a profiler: a tool that records, with timestamps, what each part of a program does over time and lays it out on a visual timeline. I annotated the loop so that each phase of a step (data loading, forward pass, backward pass, optimizer update) and each individual step appear as named, coloured bars on that timeline. The first profile, taken on the five-store layout the team had been training on, was stark. The GPUs sat idle for more than 90 per cent of the recorded window, with a single data-loading step taking about 76 seconds while they waited. A plain wall-clock log of each batch showed the same thing as a rhythm: about 12 fast batches, then 1 stall of roughly 80 seconds, repeating every 12 batches, which is the number of worker processes. The pipeline was limited by data loading, not by computation. The detailed account of why, and the experiments that pin it down, is the subject of the thesis.

Carrying the fix to production scale. The thesis establishes the fix on smaller, controlled datasets: store the same data in far fewer, larger files, so that each sample opens about 9 files instead of about 90, with the data the model sees left bit-for-bit identical. From those experiments the thesis projects that merging alone is enough to keep the GPUs fed without interruption. Taking this to the full production record was further work during the stay, and is not detailed in the thesis. Joan Vedri built the full merged store, spanning the whole time range with every input variable stacked together. Unfortunately, over the full time range the merged store did not keep the GPUs fed at all times, contrary to that projection. The periodic spikes did not disappear. They were far smaller than before, 1–2 seconds against the 80-second stalls of the original layout, but they were still there, and a run of 10 epochs took 2.3 hours, down from the tens of hours of the original layout but short of the goal. This sent us looking for the cause of the store-size effect: the larger the store, the slower each sample, even though every sample reads the same amount of data. I tracked it to xarray, the convenience layer the loader used to address the files by name, which rebuilds a large internal structure on every read whose cost grows with the size of the whole store rather than with the slice actually read. Writing a loader that reads each patch directly from the storage format, without that layer, removed the overhead and brought the run down to 1.3 hours, the point at which the GPUs, not the data, set the pace. Table 1 collects the figures. Every step was checked with the parity test and kept the data bit-identical, so the model's accuracy is unchanged, and the per-GPU speed holds as the work is spread from 1 GPU to 16.

6 Relation to the Master's Studies

The internship leaned on 2 parts of the master's degree in particular: the courses on artificial intelligence and, above all, the course on parallel programming.

Data layout	Time for 10 epochs (1 node, 12 workers)
Five separate stores (original)	about 55 hours
Merged store	about 2.3 hours
Merged store, direct reader	about 1.3 hours

Table 1: Wall-clock time to train 10 epochs of the historical AI4Land model on 1 MareNostrum 5 node, for the 3 data layouts. The merged store cuts the number of files opened per sample; the direct reader removes the remaining software overhead and makes the run limited by the GPUs. All 3 feed the model bit-identical data.

The artificial-intelligence courses are what let me understand the system I was optimizing. To change the data pipeline without changing what the model learns, I had to know what the model is and what training does to it: a neural network whose weights are fitted by minimizing a loss with stochastic gradient descent, a forward pass that turns an input into a prediction, a backward pass that computes how to adjust the weights, and an optimizer step that applies the adjustment. Knowing which of these the data layout could and could not affect is what let me argue that a faster pipeline still trains the same model.

Parallel programming is the part the internship drew on most, because the bottleneck is a parallel-computing problem. The data loader runs many worker processes that produce samples while the GPU consumes them: when the shared queue between the producers and the consumer runs dry, the consumer stalls, and that is exactly the periodic spike I had to explain. Diagnosing it meant reading the profiler timeline at the level of individual threads and processes, asking what each was doing at each instant. And training across the 4 GPUs of a node is distributed parallelism, each GPU running its own process with their results combined by a collective operation. Without the vocabulary and the mental model from that course the traces would have been unreadable; with them, I could trace the stalls to their cause and cut a training run of 55 hours down to 1.3.

I also carried over 2 working habits from the degree. One is statistical care with noisy measurements: I repeated each configuration several times, discarded the fastest and slowest readings, and judged a result by the tail of its distribution rather than the average, since the slow stalls in the tail are what set the training time. The other is the modelling cycle itself, form a hypothesis, design a test that changes a single factor, measure, and accept or reject it, which was the method of the whole internship.

7 Problems Encountered and How They Were Resolved

Maintenance windows on the supercomputer. MareNostrum 5 was undergoing migrations and maintenance during the internship, and there were stretches of hours when no job could run and no measurement could be taken. I used that time to read the literature on data loading and parallel filesystems and to prepare the exact tests to run once the machine came back, so the downtime turned into reading and planning rather than lost time.

Reproducing an earlier profiling result. The first aim of the profiling was to reproduce a slowdown a colleague had seen earlier, and at first I could not. The cause was the profiling window. The costly stall recurs every 12 batches while my window covered roughly batches 5 to 10 and so fell in the gap and saw nothing; the earlier run had profiled

batches 10 to 15 and happened to catch it. The profiler itself also perturbed the timing, shifting the stall by 1 step. It took several runs, and a careful look at a plain per-batch log, to realize that the stall was periodic at the worker count and that the window had to include it. Once the periodicity was understood, the result reproduced cleanly.

A noisy, shared machine. MareNostrum 5 is shared, so the same setup could look fast and then slow a short while later, depending on what other jobs were doing and on whether a file had been read recently. A single timing could not be trusted. I measured every configuration several times, discarded the fastest and slowest readings, and averaged the rest; and I never ran 2 measurement jobs against the same data at once, since they would distort each other.

Making the data layouts agree. Moving from the original multi-store layout to a single store changed the numbers the loader returned, which would have invalidated any speed comparison. The first sign was the training loss dropping to exactly 0 on the new layout, which I traced to missing values in the climate-zone input that the old path had filled and the new one had not. Beyond that bug, the 2 paths differed in how they normalized variables, what value they used for missing data, and how they remapped land-use labels. I reconciled them so that both paths return identical tensors, and turned that check into the parity test described above, so that from then on no comparison could silently compare different data.

8 Evaluation of My Own Work

There are 2 things I would do differently. I went deep on the environment and the tooling: a full repackaging of the codebase, the removal of many hard-coded paths and settings, and a thorough uv and ruff setup. It added real value, and the team kept it, but a lighter setup would have reached nearly the same goals, and the time it took could have gone into the profiling, which is the heart of the work. I would also have liked to understand better what happens after training, in particular how the maps the model produces are turned into the future land-use scenarios that the climate models consume; that part is downstream of my work, but understanding it well would have helped me weigh which parts of the pipeline matter most. Overall, though, I am happy with my time at BSC, and I have been told the work was profitable and useful to the AI4Land team.

9 Assessment of the Experience and Suggestions

The experience was a clear net positive. It was my first time inside a real research group, working on a production pipeline and a top-tier supercomputer, and I learned a great deal about how research is actually done day to day and what the work of a researcher looks like: the reading, the false starts, the careful measurement, and the slow narrowing toward an answer. The team was extremely welcoming, and Oriol Tintó Prims guided me along the path, available when I needed direction and giving me room to explore when I did not. I do not have substantive suggestions for improvement; the supervision and the openness of the team left me little to add.